Assignment-2 -

Ques1. Graphical User Interface (GUI) in Python -

→ A Graphical User Interface (GUI) allows users to interact with a program visually using elements like: windows, buttons, text boxes & menus. In Python, GUI programming is popular for building desktop applications & several libraries make this task easier. The most commonly used GUI libraries in Python include Tkinter, PyQt, Kivy & wxPython.

① Tkinter: The Standard GUI library

Tkinter is Python's standard GUI library & is bundled with most Python installations. It provides a fast & easy way to create GUI applications. Tkinter is based on the Tk GUI toolkit & offers a variety of widgets such as labels, buttons, entry fields & canvas to design the GUI layout.

Ex.-

```
import tkinter as tk
window = tk.Tk()
window.title("Simple GUI")
label = tk.label(window,
text = "Hello, World!")
label.pack()
button = tk.Button(window, text=
"Click Me", command = window.
 quit)
button.pack()
window.mainloop().
```

② PyQt & PySide - PyQt & PySide are powerful libraries that bind Python to the Qt application framework. They are more advanced than Tkinter & provide tools for creating complex & modern-looking GUIs. Features include support for signals & slots, multimedia, networking & more.

③ Kivy - It is an open-source Python library designed for developing multitouch applications. It works on windows, linux, macOS, Android, & iOS. Kivy is useful for applications that require gestures, animations or need to run on mobile platforms.

④ wxPython - It is a wrapper around the wxWidgets C++ library. It provides native look-and-feel on different platforms & is suitable for more complex applications.

Ques 2. Windows & Window Components -

→ Windows & window Components are In Python GUI programming, a window serves as the primary container where all (widgets) visual elements are displayed. Using GUI libraries like Tkinter, Python allows developers to create & manage windows easily.

① Main Window (Root Window) -

→ The main window, also known as root window, is the starting point of a GUI application. It is where all other components (like buttons, labels & entry fields) are placed.

Ex. Using Tkinter:

```
import Tkinter as Tk
root = tk.Tk()
root.title("My Application")
root.geometry("300X200")
root.mainloop().
```

② Window Components (Widgets) -

→ Python GUI windows contain various components, also called widgets, each designed for specific purposes:

- Label: Displays text or images.

```
label = tk.Label(root, text="Hello, World!")
label.pack().
```

- Button: Triggers & actions when clicked.

```
button = tk.Button(root, text="Click Me",
    command=root.quit)
button.pack().
```

- Entry: Allows users to input text

```
entry = tk.Entry(root)
entry.pack()
```

- Text: Multi-line text input or display

```
text = tk.text(root, height=5,
  width=30)
text.pack()
```

k button : Toggle button for options.
diobutton : Select one option from a group.
listbox : Displays a list of selectable items.

③ Geometry Management -

→ Widgets are placed using layout managers:

- pack() : Packs widgets in blocks before placing them.
- grid() : Places widgets in a table-like structure.
- place() : Places widgets at specific coordinates.

Ques3. Input & Output with Entry Fields -

→ In Python GUI programming, Entry Fields are used to accept input from the user. These fields are typically found in forms & input boxes. The most common way to use entry fields is through the Tkinter library.

① Creating an Entry Field -

→ The Entry widget in Tkinter is used for single-line text input.

Ex. -
```
import tkinter as tk
window = tk.Tk()
entry = tk.Entry(window)
entry.pack()
window.mainloop()
```

This code creates a simple window with an input field.

② Retrieving input (Getting Data) -

→ To get the input entered by the user, the .get() method is used.

```
def show_input():
    user_input = entry.get()
    label.config(text="You entered:" + user_input)
window = tk.Tk()
entry = tk.Entry(window)
entry.pack()
button = tk.Button(window, text="Submit", command=show_input)
button.pack()
label = tk.Label(window, text="")
label.pack()
window.mainloop()
```

③ Output in Entry Field -

To display output (pre-filled data) inside an entry field, the insert() method is used.

[entry.insert(0, "Default Text")]

→ This adds the text "Default Text" at the beginning of the entry field.

(4) Clearing the Entry Field -

To clear input:

entry.delete(0, tk.End).

Ques 4. Define & Using Instance Variables -

→ In Python, instance variables are variables that are defined inside a class & are unique to each object of that class. They are used to store data that is specific to a particular object. Instance variables are usually defined inside the __init() constructor method using the self keyword.

Ex. - Class student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

→ self.name and self.age are instance variables. Each object of the class student will have its own name & age.

→ Using Instance Variables -

They can be accessed & modified using the object.

Ex. - S1 = Student("Alice", 20)
    S2 = Student("Bob", 22)
    print(s1.name)
    print(s2.age)

O/P -
Alice
22

Each object maintains its own copy of instance variables, meaning changes to one object do not affect others.

→ Modifying Instance Variables -

You can modify instance variables like this:

s1.age = 21
print(s1.age).

→ Importance -

Instance variables are useful when each object needs to maintain its own state or data. For example, in a class Car, each car object can have different values for color, model & speed.

5. Other Useful GUI resources-

→ While Tkinter is the most common used GUI library in Python, there are other libraries that offer more features, customization, & support for advanced GUI applications. Below are some other useful resources for GUI development:

① Pyqt | PySide-

- are Python bindings for the Qt application framework. Qt is a powerfool toolkit for creating professional-grade applications with advanced features like support for multi-threading, multimedia & complex widgets.

② Kivy-

- Kivy is a popular library for developing multi-touch applications & is particular suitable for mobile apps as well as desktop applications.
- Key features:
  - Multitouch support
  - Highly customizable & flexible UI elements.
  - Supports animations & gestures.

③ wxPython-

- is a Python wrapper around the wxWidgets C++ library. It provides native look-and-feel applications on Windows, macOS, & linux.
- Key Features:
- Native GUI on all platforms.
- Rich set of widgets.
- Ability to create complex, platform-specific applications.
- Strong support for handling events, custom drawing & threading.

④ Flask (for Web-based GUIs)-

- While not a traditional GUI library for desktop apps. Flask is a micro-~~based~~ framework used for creating web-based applications. It can be used for building web-based user interfaces that run in browsers.

- Key Features:

Easy to get started with.

Supports HTML, CSS & Javascript to create interactive UIs.

Can be combined with other tools like Dash (for data-driven UIs).

* GUI:- Graphical User Interface:-

```
from tkinter import * keyword
root = Tk( )
root.title ("My first GUI")
root.geometry ("500 X 500 + 0 + 0")
root.mainloop ( )
```

1) On graphical User interface is a desktop interface that allows you to communicate with the computer.

2) They carry out various activities on desktop, computers, laptops and other mobile devices.

3) Text editors and other graphical user interface applications build, read, download and erase various types of data. You can also play games such as Sodoku, chess through these apps

4) Google, chrome, firefox and microsoft edge are example of graphical user interface, internet browsers.

5) We used tkinter library to create graphical user interfaces.

Write a program to create a simple GUI

```
from tkinter import * keyword
root = Tk( )
root.title ("My first GUI")
root.geometry ("500 X 500 +0 +0")

My_button = Button (root, text = "Click Me")
My_button.pack( )
root.mainloop ( )
```

Ques:- WAP to create a frame on GUI using python

```
from tkinter import*
root = TK( )
root . title ("My first GUI")
root. geometry ("500 X 500 +0 +0")
f1 = Frame (root , bg ='grey', borderwidth =20)
f1. pack (side = LEFT)

    l = label (f1, text = ' tkinter . pycharm')
    l . pack ( )

root . mainloop ( )
```

Ques:- WAP to create an entry field on GUI using python from tkinter import.

```
root = TK( )
root . title ("My first GUI")
root . geometry ("500 X 500 +0 +0")

entry = Entry (root, width = 50)
entry . pack (pad x =20 , pady =20)
def click :
lbl = lable (root, text = "successfully created", font =('arial',15)
lbl. pack( )
My_button = Button (root, text = 'Click Me', font = ('arial', 15)
            fg ='gold', bg ='black', command = click)

My button . place (x =200, y = 100)
root . mainloop( )
```

Ques:- WAP to create a button and apply font type, font size, background colour and implement command on click

```
from tkinter import*
root = TK()
root.title ("My first GUI")
root.geometry ("500X500+0+0")
def click():
lbl = label (root, text = "Successfully created", font = ('arial',15)
    lbl.pack()

My_button = Button (root, text='Click Me', font = ('arial',15), fg =
                    'gold', bg = black, command = click)

My button.place (x=200, y=300)
root.mainloop()
```